

Building Pintos

 **There is one Makefile under each lab directory (`threads/`, `userprog/`, `vm/`, `filesystem/`).** When you are working on a specific lab, you should build Pintos under its corresponding directory.

- **Lab0 will use the same directory as Lab1, i.e., `threads/`.**

 If you take a look at the content of these Makefiles, you may be surprised that they are all the same! So what's the difference when you run `make` under different lab directories?
(hint: `Make.vars`)

Now let's build the kernel from the source code supplied for Lab0.

 **If you set up your environment using docker, we assume that you are always in the container environment mounted with the Pintos directory from now on.**

1. **First, `cd` into the `threads/` directory.**
2. **Then, issue the `make` command.** This will create a build directory under `threads/`, populate it with a Makefile and a few subdirectories, and then build the kernel inside. The entire build should take less than 30 seconds.
3. **This is the end of building.**

After building, the followings are the interesting files in the `build/` directory:

- **"Makefile"**
 - A copy of `pintos/src/Makefile.build`. It describes how to build the kernel.

- **"kernel.o"**



- **Object file for the entire kernel.** This is the result of linking object files compiled from each individual kernel source file into a single object file. It contains debug information, so you can run GDB or `backtrace` (see section [Debugging](#)) on it.
- **"kernel.bin"**
 - **Memory image of the kernel**, that is, the exact bytes loaded into memory to run the Pintos kernel. This is just `kernel.o` with debug information stripped out, which saves a lot of space, which in turn keeps the kernel from bumping up against the 512 kB size limit imposed by the kernel loader's design.
- **"loader.bin"**
 - **Memory image for the kernel loader**, a small chunk of code written in assembly language that reads the kernel from disk into memory and starts it up. It is exactly 512 bytes long, a size fixed by the PC BIOS.

Subdirectories of build contain object files (.o) and dependency files (.d), both produced by the compiler. The dependency files tell `make` which source files need to be recompiled when other source or header files are changed.

Previous
Build and Run

Next
Running Pintos

Last updated 1 year ago

